Studio Nx

Backend(FastAPI/Python) 87 — AdminAuth.logout(), authenticate()

2026-10-12

Backend(FastAPI/Python) 87 — `AdminAuth.logout()`, `authenticate()` (로그아웃, 매 요청 확인)

항목	내용
파일	`studionx/backend/app/admin.py`
작업	P1-21 공지 API와 관리자 화면: `/admin` 로그아웃과 요청마다 하는 로그인 확인 (PRD 01 FR-115)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	86 `AdminAuth.login()` · 88 `DevNoteAdmin` (다음)

1. 하는 일

`AdminAuth`의 나머지 두 함수입니다. 서로 짝이라 함께 구현했습니다.

함수	언제 불리나	하는 일
`logout`	관리자 화면의 Logout 버튼	세션을 비우고 `True`
`authenticate`	`/admin`의 **모든 요청마다**	지금 요청이 로그인한 관리자의 것인지 `True`, `False`로 답합니다. `False`이면 SQLAdmin이 로그인 화면으로 보냅니다

`authenticate`가 `True`가 되는 조건은 이렇습니다.

세션과 DB의 상태	결과
세션에 `admin_user_id`가 없음 (로그인 안 함)	`False`
있고, DB에서 그 회원이 `ACTIVE`이며 `ADMIN`	`True` (세션 유지)
로그인한 뒤에 `role`을 `USER`로 바꿈 (권한 회수)	`False`, 남은 세션도 비움
로그인한 뒤에 정지(`SUSPENDED`)나 탈퇴(`DELETED`)	`False`, 세션도 비움
세션의 번호가 DB에 없음	`False`
세션의 번호가 일반 회원	`False`

2. 언제 쓰나

호출하는 곳	화면
SQLAdmin의 모든 `/admin/…` 요청 (`authenticate`), Logout 버튼 (`logout`)	관리자 화면 전체

3. 코드

`import`는 86번에서 모두 추가했습니다.

함수

```
async def logout(self, request: Request) -> bool:
    request.session.clear()
```

```
    return True


async def authenticate(self, request: Request) -> bool:
    user_id = request.session.get("admin_user_id")
    if user_id is None:
        return False
    db = SessionLocal()
    try:
        user = auth_service.get_active_user(db, user_id)
        if user.role == UserRole.ADMIN:
            return True
    except AppError:
        pass
    finally:
        db.close()
    request.session.clear()
    return False
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

문법 풀이

식	뜻
`request.session.clear()`	세션의 모든 값을 지웁니다. 이미 비어 있어도 오류가 나지 않습니다
`request.session.get("admin_user_id")`	세션에서 값을 꺼냅니다. 없으면 `None`입니다
`auth_service.get_active_user(db, user_id)`	그 번호의 회원이 아직 `ACTIVE`이면 돌려주고, 없거나 정지, 탈퇴이면 `AppError`를 던집니다. 일반 API의 재발급 확인과 같은 함수입니다
`try … except AppError: pass … finally`	못 쓰는 계정이면 오류를 삼키고, 뒤쪽의 `clear()`와 `return False`로 갑니다. `return True`가 `try` 안에 있어도 `finally`의 `db.close()`는 실행됩니다

4. 흐름

`authenticate`:

1. 세션에 관리자 번호가 없으면 바로 `False`입니다.
2. 있으면 DB 세션을 열어 그 회원이 정상 계정인지 확인합니다.
3. 정상이면서 `ADMIN`이면 `True`입니다.
4. 그 밖의 모든 경우(정지, 탈퇴, 권한 회수, 없는 번호)는 세션을 비우고 `False`입니다.

5. 왜 이렇게

선택	이유
세션만 믿지 않고 **매 요청 DB 확인**	세션 쿠키는 로그인 때의 상태를 담고 있습니다. 로그인한 뒤에 정지되거나 권한이 회수된 계정이 쿠키만으로 계속 관리자 화면을 쓰는 일을 막습니다. 요청 한 번에 쿼리 한 번(기본키 조회)이라 부담이 작습니다
`get_active_user` 재사용	정지, 탈퇴 판단이 일반 API와 같은 기준이어야 합니다

선택	이유
못 쓰는 계정이면 세션도 비움	이후 요청에서 같은 확인을 반복하지 않고, 남은 쿠키가 쓸모없게 됩니다
`logout`이 항상 `True`	로그인하지 않은 채 불러도 문제가 없고, 실패할 일이 없습니다
세션에 역할을 저장하지 않음	역할은 DB가 정한 값이어서, 매번 DB에서 읽는 것이 맞습니다

6. 시험 (`tests/test_admin_auth.py`, 이번에 8개 추가)

시험	결과
`logout`: 세션이 비워지고, 이미 비어 있어도 `True`	통과입니다
세션이 없으면 `False`	통과입니다
정상 관리자는 `True`이고 세션 유지	통과입니다
로그인한 뒤에 `role`을 `USER`로 바꾸면 다음 요청이 `False`, 세션도 비워짐	통과입니다
로그인한 뒤에 정지, 탈퇴	둘 다 `False`, 세션도 비워짐
없는 번호, 일반 회원 번호가 세션에 있음	둘 다 `False`

이 파일의 시험은 86번의 7개와 합쳐 15개이며 모두 통과했습니다. `ruff`도 통과했습니다.